Background

Burger Plus started as a tiny walk-up shack where locals could build their dream burger. Word spread quickly about the place, then people were lining up holding in their excitement for crafting their own creations at a fast rate. Over time, manually tracking orders, inventory, and staff, got too chaotic. This led to frequent mistakes, and the manager struggled to keep up with this popularity. To fix this issue, Burger Plus implemented a relational database to manage customers, menu items, and orders, providing a structure that's reliable to support the daily operations.

Operation of the Enterprise:

- Customer Management: Customers can register with personal information or order anonymously. The system records all registered customers with their registration date. (customers.py)
- Menu Management: Menu items are stored with details like name, category, price, description, and availability. Items can be marked unavailable if the ingredients run out. (menu.py)
- Order Processing: Employees create orders linked to customers (if registered) and employees. Each order records the date, time, items, quantities, unit prices, and total amount. Orders are automatically updated from "Pending" to "Complete." (orders.py)
- Data-Driven Operations: The database provides a single source of truth for customers, menu items, and orders, ensuring consistency and real-time access. (utils.py)

Justification for the Database

The database is essential for Burger Plus to manage interconnected information reliably. It maintains data consistency and accessibility, tracks registered customers, ties orders to staff for accountability, guarantees accurate orders, consistent menu availability and pricing, and facilitates expansion as the business grows. Burger Plus minimizes errors and facilitates effective, data-driven operations by utilizing the capabilities of `customers.py`, `menu.py`, and `orders.py`.

Relational Schema

1. EMPLOYEES (employee_id, first_name, last_name, email, hire_date, job_role, salary)

   Stores all employee details. Each order references an employee who handled it.

2. CUSTOMERS (customer_id, first_name, last_name, phone_number, email, registration_date)

   Stores registered customer information. Orders may reference a customer for tracking purposes.

3. MENU_ITEMS (item_id, name, category, price, description, is_available)

   Stores all products sold, including prices and availability. Connected to ORDER_DETAILS to identify items in an order.

4. INVENTORY (inventory_id, ingredient_name, unit_of_measure, current_stock, reorder_point)

   Tracks ingredient stock levels and reorder points, supporting menu-item availability.

5. ORDERS (order_id, customer_id, employee_id, order_date, order_time, total_amount, order_status)

   Stores all orders with timestamps, total amounts, and status. Links to CUSTOMERS (optional) and EMPLOYEES (required). Parent of ORDER_DETAILS.

6. ORDER_DETAILS (order_id, item_id, quantity, unit_price)

   Junction table connecting ORDERS and MENU_ITEMS, recording the quantity and unit price of each item. Primary key is (order_id, item_id).

Team Roles

Kendrick Manchester — Team Leader

- Helped manage the project and keep track of deadlines.
- Made sure team members communicated and worked together.
- Assisted wherever needed, including E-R diagram design, report writing, data generation, and testing.

Jakobe Allen — E-R Diagram Lead

- Created the E-R diagram for the database.
- Defined entities, attributes, keys, and relationships.
- Set the rules for how tables connect (cardinalities and participation).

Dillon Davis — Report Lead

- Wrote the description of Burger Plus.
- Drafted the relational schema for all six tables.
- Summarized team responsibilities.

Dennis Garay — Data Generation Lead

- Wrote scripts to fill all six tables with sample data.
- Made sure the data looked realistic for customers, employees, menu items, orders, and shifts.
- Checked that all table relationships were correct.

Austin McBurney— Testing & Instructions Lead

- Wrote five test queries covering create, read, update, delete, and joins.
- Produced sample outputs from the database.
- Wrote instructions on how to install, run, and test the database.